UNIVERSIDAD CONTINENTAL · DOCENCIA

# Manual técnico de construcción

Sala situacional de inmunizaciones del Perú

R · Shiny · golem · Versión 0.2.0 · 8 de noviembre de 2026

Reconstrucción didáctica de la arquitectura y de sus decisiones: del contrato de datos al dashboard probado en Linux, Windows y macOS.

Autoría del proyecto: Dr. Ronald Fabricio Salas Guerrero.

Repositorio: https://github.com/f-slg/sala-inmunizaciones-peru

Código verificado: 0534f358b5eab7576e9afa65b93b1cdc1c4f0472.

Este documento explica el diseño implementado. No es un registro cronológico de cada comando utilizado. Se distinguen los pasos para estudiar el proyecto existente de las instrucciones opcionales para iniciar otro proyecto vacío.

El dominio es educativo. Catálogo de vacunas, reglas clínicas, análisis administrativo y producto físico tienen responsabilidades distintas. La trazabilidad y la validación forman parte del diseño.

# Mapa de construcción

1. Delimitar las preguntas y separar responsabilidades.
2. Crear la estructura de paquete con golem.
3. Definir el contrato y sus claves.
4. Construir catálogos y demostración reproducible.
5. Validar e importar de forma transaccional.
6. Generar plantillas y diccionario.
7. Programar indicadores administrativos.
8. Evaluar dosis específicas del registro nominal.
9. Conciliar existencias y calcular necesidades.
10. Incorporar cartografía real y selector territorial.
11. Conectar la interfaz mediante reactividad y módulos.
12. Generar informes y conservar el proyecto.
13. Probar resultados y portabilidad.
14. Publicar código y preparar una extensión.
15. Practicar una modificación pequeña y comprobable.

Lea este manual con el repositorio abierto en RStudio. Las rutas indicadas son relativas a la raíz, donde se encuentran DESCRIPTION y salainmunizaciones.Rproj. Los fragmentos de consola R requieren haber ejecutado pkgload::load_all(".").

Para repetir los resultados de referencia, use el código verificado. La rama main puede contener mejoras documentales posteriores.

# 1. Delimitar preguntas y responsabilidades

La sala responde a cuatro decisiones: dónde hay brechas, qué seguimiento resulta evaluable, cuántos recursos se requieren y qué acción se evaluará. Se separó el cálculo sanitario de los controles visuales para poder comprobarlo sin navegador.

| Capa | Responsabilidad | Archivos |
|---|---|---|
| Dominio | Contrato, validación, indicadores, reglas nominales y logística. | R/domain_*.R |
| Servicios | Excel, cartografía, catálogos de entrada, demostración e informe. | R/service_*.R |
| Presentación | Entradas, tablas, mapas, gráficos y descargas. | R/mod_*.R |
| Orquestación | Estado de sesión y conexión de módulos. | R/app_server.R |
| Inicio | Configuración, UI y ejecución. | app.R, R/app_ui.R, R/run_app.R |

Una función de dominio recibe datos y devuelve resultados. Evite que lea input directamente o escriba una salida Shiny: así puede reutilizarla en scripts, pruebas y la aplicación.

El estado inicial se valida antes de crear la sesión. El modo público elimina los registros nominales de los datos iniciales y deshabilita su carga. El paquete no implementa autenticación, base de datos multiusuario ni auditoría persistente: estas capacidades requerirían diseño adicional para un uso operacional.

No confunda producto físico, antígenos, dosis y regla. Una vacuna combinada puede cubrir varios componentes, pero no debe generar varios pedidos del mismo frasco. Las rutas alternativas tampoco representan poblaciones adicionales que puedan sumarse sin conciliación.

# 2. Crear la estructura de paquete con golem

## Para estudiar esta implementación

Abra salainmunizaciones.Rproj y ejecute:

```
source("instalar.R")
pkgload::load_all(".")
shiny::runApp()
```

app.R carga la copia abierta con pkgload y devuelve run_app(). Esto evita usar sin advertirlo una versión previamente instalada del paquete.

## Para iniciar otro proyecto vacío

La documentación de golem propone crear un proyecto mediante su plantilla o create_golem(). Ejemplo opcional, en una carpeta nueva y fuera del proyecto existente:

```
install.packages("golem")
golem::create_golem("mi_sala_nueva")
```

Este comando crea un esqueleto; no reconstruye automáticamente el código ni los catálogos del presente repositorio. No lo ejecute sobre la carpeta existente.

| Pieza del paquete actual | Función |
|---|---|
| DESCRIPTION | Identidad, versión, dependencias y requisitos de R. |
| NAMESPACE y man/ | Exportaciones y ayuda generadas con roxygen2. |
| inst/extdata/ | Productos, reglas, grupos y fuentes en CSV. |
| inst/golem-config.yml | Modo público, entorno y tamaño máximo de carga. |
| inst/app/www/ | Recursos de presentación de la aplicación. |
| tests/testthat/ | Casos numéricos, adversos y de integración. |
| dev/ | Inicio, desarrollo y generación de recursos auxiliares. |

run_app() envuelve shinyApp() mediante with_golem_options(). El código de presentación se divide en UI y servidor; golem aporta la organización de paquete. La aplicación no depende de rutas personales de un sistema operativo.

Fuente de la estructura general: https://thinkr-open.github.io/golem/

# 3. Definir el contrato y sus claves

R/domain_schema.R define data_schema(): nombres exactos de hojas y columnas. empty_dataset() crea tablas vacías con ese contrato. Hay 14 tablas y 96 entradas de variables por combinación hoja/campo.

| Tabla | Grano o clave lógica |
|---|---|
| configuracion | Una configuración por proyecto. |
| territorios | ubigeo. |
| poblacion | anio + ubigeo + grupo_id. |
| vacunacion | anio + mes + ubigeo + regla_id + estrategia. |
| personas | persona_id. |
| eventos | evento_id; cada aplicación referencia persona_id. |
| elegibilidad | Persona, grupo y episodio con intervalo documentado. |
| programacion | anio + ubigeo + regla_id. |
| inventario | ubigeo + producto_id + lote. |
| movimientos | movimiento_id; referencia el lote. |
| presentaciones | producto_id. |
| consumo | Año, mes, territorio y producto. |
| cadena_frio | ubigeo. |
| plan_mejora | accion_id. |

configuracion, poblacion, territorios y vacunacion son hojas obligatorias. optional_sheets() identifica las otras diez. Una hoja opcional no significa que sus filas puedan omitir campos necesarios para ese cálculo.

Las relaciones se expresan con identificadores, no con posiciones de fila. Los ubigeos son texto de seis caracteres. Las personas usan identificadores seudonimizados; los eventos almacenan una fila por aplicación para admitir esquemas extensibles sin añadir una columna por vacuna.

Antes de añadir una variable, defina su unidad, grano, valores válidos, sensibilidad y efecto sobre las claves. Si cambia el contrato, actualice también diccionario, importación, demostración, plantillas, pruebas y documentación.

# 4. Construir catálogos y demostración reproducible

load_catalog() lee productos.csv, reglas.csv, grupos.csv y fuentes.csv desde inst/extdata/. Los CSV son el catálogo del programa; las hojas cat_* del XLSX son copias consultables y no modifican el motor.

El catálogo de esta versión contiene 33 productos y 116 reglas. Solo 34 reglas tienen evaluación nominal educativa automatizada. Los estados y fuentes permiten distinguir revisión, condicionalidad y alcance. No transforme un estado en verificada solo para que un caso sea calculable.

R/service_demo.R contiene demo_dataset(). Genera datos sanitarios simulados para tres distritos reales: Lircay 090301, Anchonga 090302 y Callanmarca 090303. La demostración permite repetir cálculos, probar cargas y explicar ausencias de reporte.

```
pkgload::load_all(".")
catalogo <- load_catalog()
datos <- demo_dataset(catalogo)
revision <- validate_dataset(datos, catalogo)
stopifnot(!any(revision$nivel == "Error"))
```

Los reportes administrativos completos llegan hasta octubre de 2026. Los meses posteriores conservan ausencia y estado pendiente. El conjunto tiene fuentes marcadas como simuladas y no representa la situación epidemiológica de esos distritos.

Para añadir una vacuna, revise producto físico, componentes, regla, grupo, ruta, intervalos, edades, vigencia y fuente. Una fila de catálogo no basta para automatizar una historia clínica; los casos de transición o elegibilidad especial necesitan reglas y validación documentadas.

La referencia normativa incluida en el proyecto es NTS 246 y RM 561-2026. Este manual describe cómo el código utiliza el catálogo; no constituye una revisión clínica independiente de todas las disposiciones ni sustituye la verificación de modificaciones posteriores.

# 5. Validar e importar de forma transaccional

R/service_excel.R implementa import_workbook(). Su secuencia es:

1. Leer las hojas y normalizar sus nombres.
2. Rechazar nombres duplicados y comprobar hojas obligatorias.
3. Leer columnas como texto y retirar filas completamente vacías.
4. Convertir números, fechas y lógicos con coerce_dataset().
5. Ejecutar validate_dataset() del dominio.
6. Devolver exito, datos e informe; datos es NULL si hay errores.

```
resultado <- import_workbook("entregables/Demostracion_integral.xlsx")
stopifnot(resultado$exito)
head(resultado$informe)
```

En mod_workspace_server(), el estado solo se reemplaza cuando exito es TRUE. Así, una carga inválida no deja mezclados datos anteriores y nuevos. Los avisos se muestran junto a los errores, pero tienen significado distinto.

La validación comprueba estructura, referencias, duplicados, valores, fechas y coherencia entre tablas según las reglas implementadas. La calidad analítica añade avisos por reportes incompletos y coberturas mayores de 100 %.

Las fechas admiten tipo Date, fechas Excel y cadenas ISO. Los lógicos aceptan representaciones controladas de verdadero/falso. meta se convierte a número en programacion, pero permanece texto en plan_mejora; esta distinción evita perder metas como “95 %” al reimportar.

Nunca modifique el estado antes de terminar la validación. Si se incorpora una nueva importación CSV o API, debe producir el mismo contrato y pasar por el validador. No salte esa capa para simplificar una carga.

# 6. Generar plantillas y diccionario

R/domain_catalogues.R implementa territory_catalog(), entry_options() y variable_dictionary(). El diccionario vincula hoja y variable e incluye tipo, unidad, obligatoriedad, hoja opcional, definición, valores, ejemplo y sensibilidad.

R/service_catalogues.R agrega hojas de catálogo, rangos nombrados y controles. R/service_excel.R conserva el generador integral. Ejemplo en la consola R:

```
write_template(tempfile(fileext = ".xlsx"))
write_template(tempfile(fileext = ".xlsx"), example = TRUE)
write_population_template(tempfile(fileext = ".xlsx"))
write_catalogue(tempfile(fileext = ".xlsx"))
```

Las listas usan dataValidation estándar de OOXML y rangos como lista_ubigeo o lista_regla_id. Se evitó depender de validaciones x14 específicas de extensiones de Excel. Los códigos se formatean como texto y los conteos tienen controles numéricos. En plantillas vacías, las validaciones llegan hasta la fila 5001.

El ubigeo utiliza advertencia, no bloqueo estricto, para permitir un código INEI verificado de creación posterior que aún no figure en la cartografía. El resto de listas exige códigos del catálogo. La validación de R sigue siendo necesaria, ya que pegar datos puede eludir los controles de Excel.

save_sala_workbook() elimina relaciones de dibujo e impresión sin destino que openxlsx puede crear en hojas vacías. Se verificó la consistencia de las relaciones OOXML de los libros generados.

La calculadora independiente conserva sus fórmulas y cachés recalculadas; no es una entrada para import_workbook(). Si cambia sus fórmulas, recalcule con un motor de hoja de cálculo y revise los resultados antes de distribuirla. Una lectura/escritura con una biblioteca no garantiza recalcular fórmulas.

# 7. Programar indicadores administrativos

R/domain_indicators.R implementa coverage_table(), coverage_summary(), dropout_table() y monthly_trend(). coverage_table() filtra año, meses, territorio y estrategia regular; busca el denominador del grupo asignado a cada regla.

El acumulado exige meses completos desde enero hasta el corte, dosis finitas y estado completo. Si falta reporte, mantiene NA. La función no reemplaza ausencias por cero.

```
x <- coverage_table(datos, year = 2026, month = 10)
z <- x[x$regla_id == "PCV_3" & x$ubigeo == "090301", ]
stopifnot(z$dosis == 90, z$denominador == 120)
stopifnot(z$cobertura == 75)
s <- coverage_summary(x[x$regla_id == "PCV_3", ])
stopifnot(abs(s$cobertura - 170 / 270 * 100) < 1e-8)
```

coverage_summary() agrupa dentro de una misma regla. La operación correcta es suma de dosis / suma de denominadores; no media simple de porcentajes. Las poblaciones de distintos grupos o reglas no se agregan como personas únicas.

Brecha a 95 % = max(0, ceiling(denominador × 0.95) − dosis). La brecha es administrativa y no identifica a cada persona pendiente.

La deserción usa (primera − última) / primera × 100. Si resulta negativa, genera una interpretación de revisión de comparabilidad. Con denominador cero no se fuerza un porcentaje.

monthly_trend() comprueba la presencia de cada territorio seleccionado en cada mes. Un mes incompleto conserva NA; el acumulado posterior sigue afectado por la falta previa. Una gráfica puede describir dirección sin atribuir una causa. Las fichas de indicadores en R/domain_fichas.R documentan fórmula, unidad, fuente y limitación.

# 8. Evaluar dosis específicas del registro nominal

R/domain_nominal.R contiene assess_nominal() y los resúmenes de cumplimiento, oportunidad y necesidades nominales. Se utilizan personas, eventos, elegibilidad y reglas activas para el corte.

El diseño contrasta dosis específicas y sus antecedentes. No cuenta simplemente “tres aplicaciones” como esquema completo: una dosis posterior no sustituye otra faltante, y una aplicación después del corte se excluye.

```
evaluacion <- assess_nominal(datos, catalogo,
  cutoff = as.Date("2026-11-08"))
resumen_nominal <- nominal_summary(evaluacion)
oportunidad <- opportunity_table(evaluacion)
pendientes <- nominal_dose_needs(evaluacion)
```

Consulte la firma de assess_nominal() en el código antes de extender parámetros. Las rutas distinguen penta_ipv y hexavalente; las historias mixtas o no modeladas mantienen revisión.

Recibida, válida y oportuna son dimensiones diferentes. La tolerancia de oportunidad de 30 días es pedagógica y configurable en la función de evaluación. Las ventanas que requieren información horaria o revisión clínica no se certifican con una fecha calendario.

El estado de completo requiere que las dosis evaluadas sean válidas y ninguna requiera revisión. No lo presente como una certificación universal del esquema nacional. Las reglas especiales y condicionales quedan fuera de una automatización clínica completa.

No sume pendientes nominales a las necesidades poblacionales sin verificar que no están incluidos ya en recuperación. La existencia de registros seudonimizados tampoco elimina su sensibilidad: los eventos y fechas siguen siendo datos privados.

Pruebas relevantes: tests/testthat/test-nominal.R. Revise los casos de sustitución de dosis, fecha de corte, intervalos y rutas alternativas antes de cambiar el motor.

# 9. Conciliar existencias y calcular necesidades

R/domain_logistics.R separa inventory_reconciliation(), apply_inventory_balances(), plan_requirements(), cold_chain_summary() y consumption_forecast(). La interfaz utiliza el saldo conciliado para el pedido.

```
corte <- as.Date("2026-11-08")
saldos <- inventory_reconciliation(datos$inventario,
  datos$movimientos, cutoff = corte)
inv <- salainmunizaciones:::apply_inventory_balances(
  datos$inventario, saldos)
necesidades <- plan_requirements(datos$programacion, inv,
  datos$presentaciones, catalogo, cutoff = corte)
```

apply_inventory_balances() es interna. El ejemplo la usa tras cargar el paquete con pkgload, que prepara el espacio de nombres. No copie su lógica por posición: la conciliación ordena lotes por vencimiento y los saldos se reúnen por ubigeo, producto y lote.

Por fila de programación: administrar = ceiling(población × meta) + recuperación; brutas = administrar / (1 − pérdida). Se agrupa por año, territorio y producto antes de redondear las brutas y los frascos.

Reserva, tránsito, horizonte y reposición deben ser coherentes entre reglas de un mismo producto. El pedido neto descuenta stock utilizable y tránsito confirmado. El stock que vence antes del fin del horizonte se excluye conservadoramente y se muestra en riesgo.

Las presentaciones definen dosis por frasco, volumen empacado e insumos. Los días de existencia suponen demanda uniforme. El resumen de frío contrasta volumen del pedido y capacidad libre declarada.

La estimación por consumo requiere seis meses completos anteriores al mes del corte y usa días con stock. No representa demanda no atendida ni se suma automáticamente a la estimación poblacional. Los casos adversos se comprueban en test-logistics.R.

# 10. Incorporar cartografía real y selector territorial

R/service_map.R convierte mapsPERU::map_DIST a sf, verifica su sistema de coordenadas y transforma a EPSG:4326. Conserva ubigeo y nombres reales. La geometría se almacena en caché dentro del proceso.

map_join() une una tabla de una sola regla mediante ubigeo; informa códigos sin geometría y rechaza ubigeos duplicados. build_leaflet() crea polígonos, etiquetas y ventanas con valores. OpenStreetMap es un fondo opcional; sin él se conservan los límites locales.

```
x <- coverage_table(datos, year = 2026, month = 10)
z <- x[x$regla_id == "PCV_3", ]
mapa <- build_leaflet(z, tiles = FALSE)
mapa
```

La cobertura utiliza clases absolutas: menor de 80, 80 a menos de 95, 95 a 100, mayor de 100 y sin dato. Esto evita que el mejor distrito de un grupo con coberturas bajas aparezca verde por una escala relativa. La brecha usa una escala distinta.

R/mod_territory_picker.R implementa departamento → provincia → selección múltiple de distritos. La selección se acumula sin duplicar códigos. El fondo departamental y marcadores facilitan reconocer distritos pequeños separados por grandes distancias.

Las descargas precargan solo territorios. No crean población ni reportes sanitarios. La selección para plantillas y los filtros del conjunto cargado son estados distintos.

mapsPERU 2.0.1 aporta 1.891 distritos; no es un servicio INEI en tiempo real. Si se actualiza la cartografía, verifique CRS, claves, duplicados, geometrías válidas y cambios de ubigeo antes de reemplazar el servicio.

Fuente del paquete: https://CRAN.R-project.org/package=mapsPERU

# 11. Conectar interfaz, reactividad y módulos

app_ui() define navegación, barra lateral y 13 pestañas. app_server() valida los datos iniciales, crea state con reactiveVal() y conecta los módulos. Cada instancia de sesión mantiene su propio estado en memoria.

El flujo principal es datos → validación → estado → filtros → cálculos reactivos → módulos → exportación. Cambiar una entrada invalida las expresiones dependientes; el cálculo se vuelve a ejecutar cuando la salida lo necesita.

Un módulo usa NS(id) en su UI y moduleServer() en su servidor para aislar identificadores. El patrón de tabla compartido del proyecto permite añadir vistas sin copiar formato:

```
# En app_ui(), dentro de una pestaña o card:
mod_table_ui("mi_vista", "Mi tabla de aprendizaje")
# En app_server(), después de definir coverage:
mod_table_server("mi_vista", coverage)
```

No pegue esas llamadas aisladas en la consola esperando una aplicación nueva: deben incorporarse a las funciones correspondientes. Para un módulo especializado, revise mod_coverage.R o mod_territory_picker.R y conserve un único par de UI/servidor con ids coincidentes.

app_server() delimita los datos por ubigeo y vincula eventos y elegibilidad con las personas seleccionadas. La cobertura mantiene su filtro administrativo; el registro nominal utiliza el corte exacto. Los parámetros logísticos se obtienen del libro cargado y del corte de inventario.

Los cambios de alumno o plan modifican state; los filtros deben conservarse cuando las opciones siguen siendo válidas. Las pruebas de módulos protegen ese comportamiento.

Conceptos base de UI, servidor y reactividad:
https://shiny.posit.co/r/getstarted/shiny-basics/lesson1/

# 12. Generar informes y conservar el proyecto

R/mod_reports.R expone descargas Excel y HTML. app_server() reúne tablas agregadas de cobertura, población, necesidades, inventario, frío, calidad, plan y trazabilidad. No añade tablas de personas ni eventos a ese conjunto.

export_aggregates() y report_html() rechazan columnas nominales identificadoras previstas, como persona_id y fecha_nacimiento. El HTML escapa textos aportados por el usuario para no interpretarlos como código.

La trazabilidad incluye versión de aplicación, huella MD5 del conjunto, corte administrativo, corte nominal y fuentes. La huella permite comparar conjuntos; no cifra ni anonimiza el contenido.

El proyecto privado se guarda con write_template(dataset = state()). Al reimportarlo debe recuperar los datos y conservar campos como meta textual del plan. No hay guardado automático en disco, base de datos ni recuperación de una sesión perdida.

Agregado no garantiza anonimización: una descripción libre, nombre del responsable o celda pequeña puede revelar información. La revisión del contenido sigue siendo necesaria antes de compartir el informe.

## Modos de ejecución

```
# Sesión local privada, con demostración:
run_app(public_mode = FALSE)
# Consulta pública sin carga ni seguimiento nominal:
run_app(public_mode = TRUE)
```

El modo público es una restricción funcional, no autenticación ni un sistema de permisos completo. Para desplegar, prepare una carpeta separada con datos agregados revisados y excluya cualquier libro nominal. publicar.R contiene instrucciones y no despliega automáticamente.

No incluya tokens en scripts ni suponga que un repositorio público equivale a una aplicación web alojada. El servidor que ejecuta R es el lugar donde se procesan los datos.

# 13. Probar resultados y portabilidad

## Comandos desde la raíz

```
pkgload::load_all(".")
testthat::test_local(".")
```

En una terminal, desde el directorio del proyecto:

```
R CMD build .
R CMD check --no-manual salainmunizaciones_0.2.0.tar.gz
```

Las pruebas deben contrastar resultados y riesgos reales: denominadores duplicados, periodos incompletos, dosis futuras, secuencias, rutas alternativas, stock ordenado por vencimiento, parámetros inconsistentes y reimportación de Excel.

| Archivo de pruebas | Riesgo principal |
|---|---|
| test-indicators.R | Cálculos administrativos y reporte incompleto. |
| test-nominal.R | Dosis específicas y límites del motor. |
| test-logistics.R | Redondeos, saldo, vencimiento y necesidades. |
| test-validation-excel.R | Contrato, carga y exportación. |
| test-map.R | Geometría y unión por ubigeo. |
| test-modules.R | Estado y comportamiento de módulos. |
| test-catalogues.R | Diccionario, listas, territorio y persistencia de meta. |

La versión de código 0534f35 aprobó 139 comprobaciones por sistema con R 4.6.2 en Linux, Windows y macOS. La comprobación local adicional con R 4.5.2 terminó con Status: OK.

El navegador comprobó selección de Chachapoyas, Arequipa y Lima, mapa, descarga con tres ubigeos y tablas sanitarias vacías, sin errores JavaScript ni Shiny. Las pruebas de R no sustituyen esta revisión visual.

La matriz está en .github/workflows/R-CMD-check.yaml. Windows utiliza el R preinstalado del runner; Linux y macOS preparan R con setup-r. El proyecto no fija todas las versiones mediante un lockfile: registre sessionInfo() y revise compatibilidad al actualizar dependencias.

# 14. Publicar código y preparar una extensión

El repositorio es público y utiliza licencia MIT para el código del proyecto. Las dependencias, la cartografía y los materiales de terceros conservan sus propias licencias. No interprete MIT como permiso para publicar cualquier fuente sanitaria.

.gitignore excluye carpetas privadas y libros no autorizados, con excepciones explícitas para cinco XLSX de distribución. Esa configuración reduce errores accidentales; no elimina información sensible ya incluida en un archivo autorizado o en el historial.

Antes de proponer un cambio, revise git status y git diff, confirme que no agregó datos personales ni credenciales y ejecute las pruebas del componente. Si modifica dominio, contrato o importación, revise también sus consumidores y la demostración.

Para publicar una app, se requiere un servicio que ejecute R/Shiny. El repositorio no incluye autenticación, persistencia ni configuración universal del alojamiento. Siga publicar.R y las condiciones del entorno que elija.

## Extensiones que requieren diseño adicional

| Extensión | Trabajo necesario |
|---|---|
| Fuente HIS o API | Adaptador al contrato, trazabilidad, permisos y controles de completitud. |
| Nuevas vacunas o cohortes | Reglas, fuentes, estados, elegibilidad y pruebas de casos clínicos límite. |
| Nuevos distritos | Cartografía, CRS, versión territorial y concordancia histórica. |
| Multiusuario persistente | Autenticación, almacenamiento, aislamiento y auditoría. |
| Alertas o envíos | Regla explícita, destinatario autorizado y registro de ejecución. |

Una extensión es revisable cuando tiene definición, datos de ejemplo simulados, resultado esperado, validación y documentación. Mantenga el principio de una persona o aplicación por fila y las claves documentadas al incorporar nuevas fuentes.

# 15. Práctica: reproducir y extender con control

## Parte A. Reproducir sin editar el motor

Ejecute el cálculo del capítulo 7 y explique por qué el agregado es 62.96 %. Exporte una plantilla para dos distritos y compruebe que no contiene población inventada:

```
g <- salainmunizaciones:::territory_catalog()
elegidos <- g[g$ubigeo %in% c("010101", "150101"), ]
archivo <- tempfile(fileext = ".xlsx")
write_template(archivo, territories = elegidos)
stopifnot(nrow(readxl::read_excel(archivo,
  sheet = "poblacion")) == 0)
```

territory_catalog() es interna; el prefijo ::: se usa aquí para estudiar el paquete, no como una API estable de integración externa.

## Parte B. Añadir una vista de aprendizaje

1. Cree una rama de trabajo y guarde el estado inicial del código.
2. Añada la tabla del capítulo 11 a una pestaña de prueba con id único.
3. Inicie la aplicación y cambie distrito y regla para comprobar su actualización.
4. Evite duplicar las fórmulas en el módulo: reciba la tabla reactiva ya calculada.
5. Documente qué pregunta responde la vista y sus límites.
6. Retire la vista de prueba o incorpórela con documentación y validación adecuadas.

## Evidencias de aprendizaje

Entregue un esquema de capas, las claves de cuatro tablas relacionadas, un cálculo reproducido, una carga rechazada y corregida, y una modificación explicada con su resultado esperado.

No cambie la norma ni el estado de las reglas como ejercicio de estilo. Un cambio de interfaz reversible puede revisarse visualmente; un cambio de fórmula o elegibilidad exige evidencia y casos de prueba apropiados.

## Fuentes y registro verificable

Código: https://github.com/f-slg/sala-inmunizaciones-peru

Pruebas: https://github.com/f-slg/sala-inmunizaciones-peru/actions/runs/42482417805

Marco golem: https://thinkr-open.github.io/golem/

Shiny: https://shiny.posit.co/r/getstarted/shiny-basics/lesson1/

Cartografía: https://CRAN.R-project.org/package=mapsPERU